

CHAPTER 15

15 - 장바구니 · 주문 · 결제 (트랜잭션)



Node.js · Express · NestJS 강의 · 2026

지금까지 상품을 만들고 보여 줬다. 이 장에서는 사고판다. 구매자가 장바구니에 담고, 주문하고, (모의) 결제한다. 여기서 게 시판으로 다루지 않은 개념이 처음 나온다 — 트랜잭션.

주문 한 건은 세 가지 일을 한 묶음으로 한다. **재고 차감** → **주문 생성** → **장바구니 비우기**. 중간에 하나라도 틀어지면 전부 되 돌려야 한다. 재고만 줄고 주문은 안 생기면 큰일이다.

이 장에서 달라진 것 (← 14장 대비)

구분	파일 / 내용
신규	<code>src/cart/</code> (장바구니) · <code>src/orders/</code> (주문·결제)
수정	<code>prisma/schema.prisma</code> — CartItem · Order · OrderItem · Payment · OrderStatus enum 추가, <code>User·Product</code> 에 역참조 · <code>app.module</code> 에 <code>CartModule</code> · <code>OrdersModule</code>
의존성	없음(기존 의존성으로 충분)
엔드포인트	<code>+ /cart</code> (담기·조회·수정·삭제) · <code>+ /orders/checkout</code> · <code>+ /orders/:id/pay</code> · <code>+ /orders/:id/cancel</code> <code>+GET /orders</code>

0. 사전 준비 — PostgreSQL + Redis

PostgreSQL과 Redis(14장에서 Homebrew로 설치)가 떠 있어야 한다.

```
brew services list          # redis 가 started 인지 확인 (아니면 brew services start redis)
```

1. 모듈 생성 (새 패키지는 없다)

```
nest g module cart    && nest g service cart --no-spec    && nest g controller cart --no-spec
nest g module orders && nest g service orders --no-spec && nest g controller orders --no-spec
```

2. 스키마 — 거래 모델 + 마이그레이션

주문 상태를 enum으로 정의하고, 거래 모델 네 개를 더한다. `User` · `Product` 에는 역참조 배열이 생긴다.

```

enum OrderStatus {
    PENDING    // 주문 생성, 결제 대기
    PAID       // 결제 완료
    SHIPPED    // 배송 중 (16장)
    DONE       // 배송 완료 (16장)
    CANCELLED  // 취소(재고 복구됨)
}

model User {
    // ... 기존 필드 그대로 ...
    orders    Order[]
    cartItems CartItem[]
}

model Product {
    // ... 기존 필드 그대로 ...
    orderItems OrderItem[]
    cartItems  CartItem[]
}

model CartItem {
    id          Int      @id @default(autoincrement())
    user        User     @relation(fields: [userId], references: [id], onDelete: Cascade)
    userId      Int
    product     Product @relation(fields: [productId], references: [id], onDelete: Cascade)
    productId   Int
    quantity    Int

    // 같은 상품은 장바구니에 한 줄만. 또 담으면 수량을 합친다(upsert).
    @@unique([userId, productId])
}

model Order {
    id          Int      @id @default(autoincrement())
    buyer       User     @relation(fields: [buyerId], references: [id])
    buyerId     Int
    status      OrderStatus @default(PENDING)
    totalPrice  Int
    items       OrderItem[]
    payment     Payment?
    createdAt   DateTime @default(now())
}

model OrderItem {
    id          Int      @id @default(autoincrement())
    order       Order    @relation(fields: [orderId], references: [id], onDelete: Cascade)
    orderId     Int
    product     Product @relation(fields: [productId], references: [id])
    productId   Int
    quantity    Int
    unitPrice   Int // 주문 시점 가격 스냅샷
}

```

```

}

model Payment {
  id      Int      @id @default(autoincrement())
  order   Order     @relation(fields: [orderId], references: [id], onDelete: Cascade)
  orderId Int        @unique
  amount  Int
  method  String
  paidAt  DateTime @default(now())
}

```

```

npx prisma migrate dev --name add-cart-orders

```

3. 코드 작성 순서

순 서	파일	내용
1	src/cart/dto/*.ts	productId · quantity 검증
2	src/cart/cart.service.ts	upsert(같은 상품은 수량 합산) + 합계 계산
3	src/cart/cart.controller.ts	/cart 담기·조회·수정·삭제 — 전부 로그인 필요
4	src/orders/orders.service.ts	checkout 트랜잭션(조건부 재고 차감 → 주문 생성 → 장바구니 비우기) · pay(모의) · cancel(재고 복구)
5	src/orders/orders.controller.ts	POST /orders/checkout · POST /orders/:id/pay · POST /orders/:id/cancel · GET /orders

각 파일의 코드와 설명은 해설서 15장에 있다.

4. 실행 + 동작 테스트 (curl) — 담기부터 결제까지

```

npm run start:dev

```

```

BASE=http://localhost:3000
login() { curl -s -X POST $BASE/auth/login -H "Content-Type: application/json" \
  -d "{\"email\":\"$1\",\"password\":\"$2\"}" | jq -r .access_token; }
ADMIN=$(login admin@example.com admin123)

# 구매자 가입 + 로그인
curl -s -X POST $BASE/auth/register -H "Content-Type: application/json" \
  -d '{"email":"b@example.com","password":"secret123","name":"구매자"}'
B=$(login b@example.com secret123)

# 관리자가 재고 3짜리 상품 등록
curl -s -X POST $BASE/products -H "Authorization: Bearer $ADMIN" -H "Content-Type:
application/json" \
  -d '{"name":"무선 이어폰","description":"x","price":89000,"stock":3,"categoryIds":[1]}'

# 1) 장바구니에 2개 담기 → 조회(합계)
curl -s -X POST $BASE/cart/items -H "Authorization: Bearer $B" -H "Content-Type:
application/json" \
  -d '{"productId":1,"quantity":2}'
curl -s $BASE/cart -H "Authorization: Bearer $B"

# 2) 주문 - 트랜잭션으로 재고 차감(3→1) + 주문 생성 + 장바구니 비우기
curl -s -X POST $BASE/orders/checkout -H "Authorization: Bearer $B"

# 3) 결제(모의) → PAID
curl -s -X POST $BASE/orders/1/pay -H "Authorization: Bearer $B" -H "Content-Type:
application/json" \
  -d '{"method":"CARD"}'

# 4) 재고를 넘겨 주문하면 → 409 + 롤백(재고 그대로)
curl -s -X POST $BASE/cart/items -H "Authorization: Bearer $B" -H "Content-Type:
application/json" \
  -d '{"productId":1,"quantity":5}'
curl -i -X POST $BASE/orders/checkout -H "Authorization: Bearer $B"      # 409
curl -s $BASE/products/1 | jq .stock                                     # 여전히 1

```

핵심 개념

트랜잭션 — 전부 되거나, 전부 안 되거나

```
return this.prisma.$transaction(async (tx) => {
  for (const item of cart) {
    // "재고가 수량 이상일 때만" 줄인다 (조건부 차감)
    const updated = await tx.product.updateMany({
      where: { id: item.productId, stock: { gte: item.quantity } },
      data: { stock: { decrement: item.quantity } },
    });
    if (updated.count === 0) throw new ConflictException("재고 부족"); // → 전체 롤백
    // ...
  }
  const order = await tx.order.create({ ... });
  await tx.cartItem.deleteMany({ where: { userId } });
  return order;
});
```

- 도중에 예외가 나면 `$transaction` 이 이미 한 작업까지 전부 되돌린다. 그래서 "재고만 줄고 주문은 없는" 상태가 생기지 않는다.
- 재고 차감을 먼저 조회→비교→차감 이 아니라 조건부 업데이트(`where: { stock: { gte: item.quantity } }`)로 한다. 두 주문이 동시에 들어와도 초과 판매가 안 된다.
- `unitPrice` 에 주문 시점 가격을 베껴 둔다. 나중에 상품 값이 바뀌어도 지난 주문 금액은 그대로다.

모의 결제와 상태

`POST /orders/:id/pay` 는 실제 PG 연동 없이 결제 기록을 남기고 상태를 `PENDING` → `PAID` 로 바꾼다. 이미 결제된 주문에 또 결제하면 409다. 취소(`/cancel`)는 결제 전(`PENDING`)만 가능하고, 차감했던 재고를 되돌린다.

핵심 파일

파일	역할
<code>orders/orders.service.ts</code>	주문 트랜잭션(조건부 재고 차감), 결제, 취소+재고 복구
<code>cart/cart.service.ts</code>	장바구니 upsert(수량 합산), 합계 계산

따라잡기 — 완성본으로 실행 (보조 경로)

수업의 기본 경로는 위처럼 직접 만드는 것이다. 따라오다 놓쳤다면, 수업 저장소의 15/ 폴더(완성 코드)로 이 장의 끝 상태에서 다시 시작할 수 있다.

```
cd 15
npm install
cp .env.example .env          # 사용자·비밀번호를 본인 PostgreSQL 값으로 수정
npx prisma migrate dev --name init  # 이 폴더 전용 DB(nest_shop_15) - 자동 생성이 안 되면
DBBeaver로 먼저 만든다
npm run seed
npm run start:dev
```

다음 장

- 16 - 상품 이미지 + 배송 상태머신: 상품에 이미지를 올리고, 주문을 PAID → SHIPPED → DONE 순서로만 넘기는 상태머신을 더한다.